

# Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO  
a.a 2025/2026  
Prof. Alessandro Ricci

## [module-4.3] SERVICE-ORIENTED COMPUTING - AN OVERVIEW

# SUMMARY

- Service-Oriented Architectures
  - SOA based on Web Services
  - REST Web Services
  - API
- From Services to Microservices
- From Services to X-AS-A-Service
- Beyond the Overview

# SERVICE-ORIENTED ARCHITECTURES (SOA)

# A WORLD OF SERVICES

- *Services* are nowadays the main approach for designing and building Internet-based distributed systems
- Service general definition
  - a service is a ***software component provided through a network-accessible endpoint***
- Two main “competing & converging” views:
  - **SOA based on “Big” Web services** (WS-\* stack)
    - conceived for enterprise-level applications
  - **REST-full web services**
    - resources exposed as (web) services
    - more and more adopted also at the enterprise level and for SOA
  - **microservices**

# SERVICE-ORIENTED ARCHITECTURES (SOA)

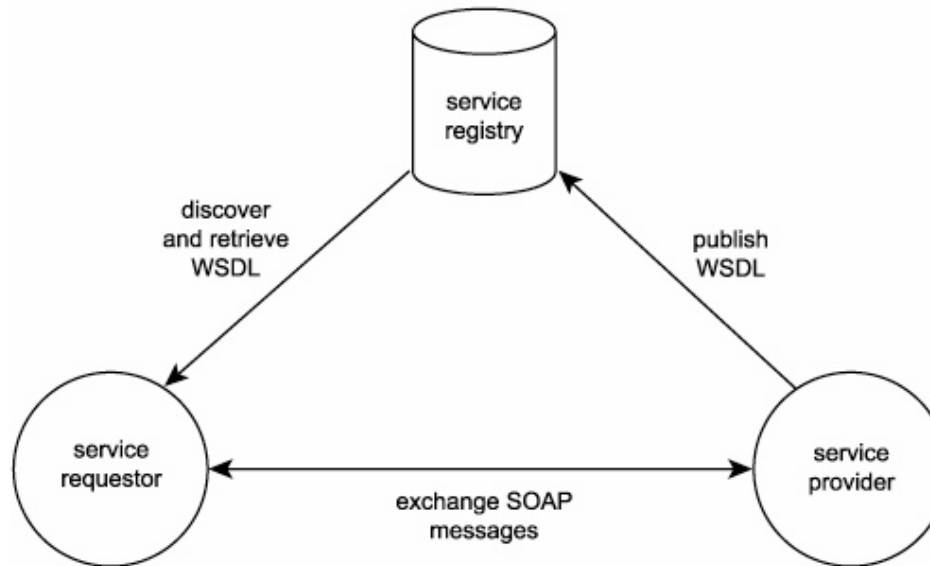
- *“Because the term ‘service-oriented’ has existed for some time, it has been used in different contexts and for different purposes. One constant through its existence has been that it represents a distinct approach for **separating concerns**. What this means is that logic required to solve a large problem can be better constructed, carried out, and managed if it is decomposed into a collection of smaller, related pieces. Each of these pieces addresses a concern or a specific part of the problem. [...] What distinguishes the service-oriented approach to separating concerns is the manner in which it achieves separation.”*

*(Thomas Erl, Service-Oriented Architecture Concept, Technology, and Design)*

# SOA

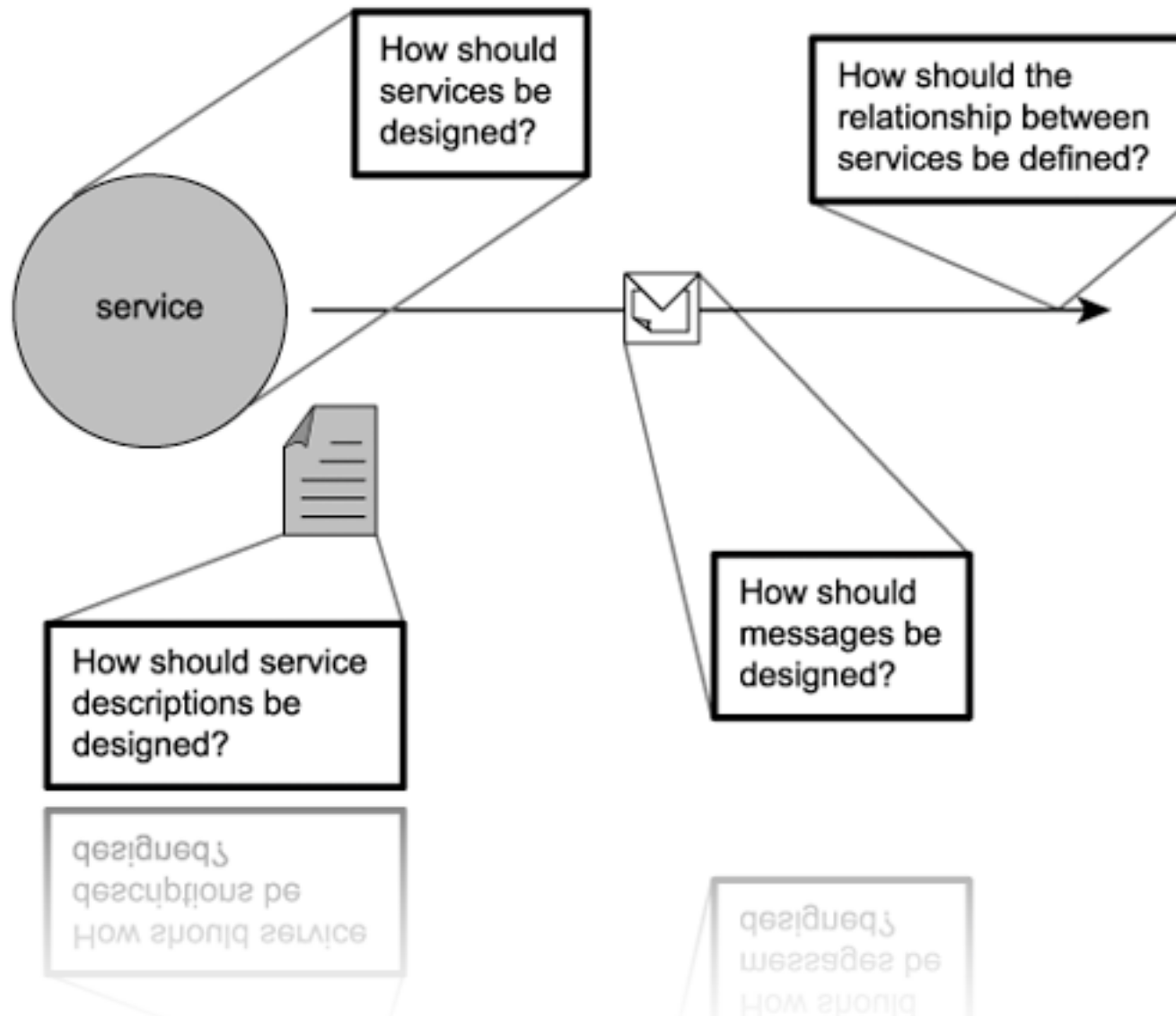
- A model in which business logic is decomposed into smaller, distinct units of (business) logic
  - collectively these units comprise a large piece of business logic, logically & physically distributed
- *Self-governance* and *autonomy* of individual services
  - independence between services
  - possible heterogeneity of implementation languages & technologies
- *Interoperability*
  - shared conventions, standards
  - key role of *interfaces*
  - contracts ruling consumer and producers of services

# SERVICE DESCRIPTION, MESSAGES



- A service is provided by some service provider, *consumed* by service consumers or requestors
- A service has a description
  - to be discovered
  - functioning as a contract
- Services relate using *messages*
  - we need messaging framework
  - message as “independent units of communication”

# SERVICE DESIGN

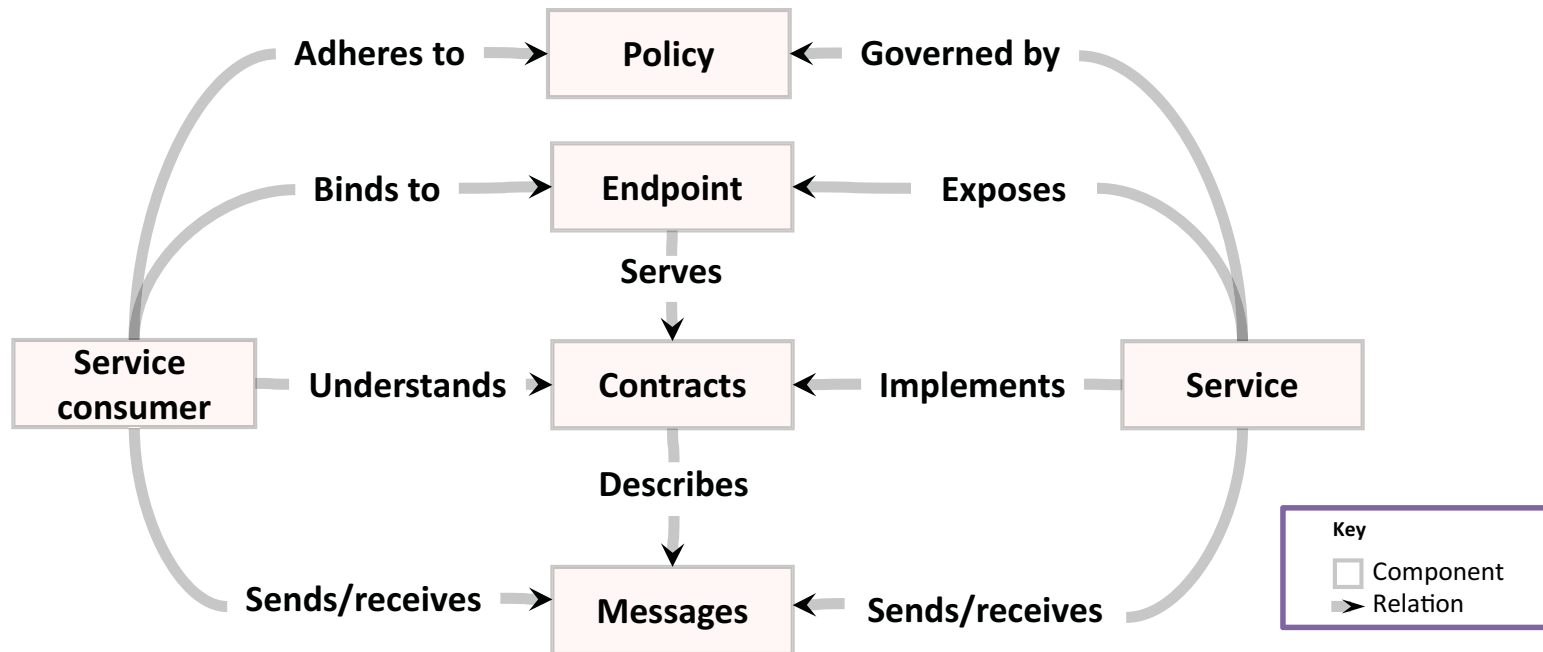




# SOA AS AN ARCHITECTURAL STYLE

- **SOA** is an *architectural style*
  - i.e. it defines components, relationships, and constraints about each component's usage and interactions
- Components in the SOA style:
  - **service, endpoint, message, contract, policy, and service consumer**
  - besides components => interactions that the components can have

# SOA COMPONENTS



**Figure 1.1** Apart from the obvious component (the service), SOA has several other components, such as the contract that the service implements, endpoints where the service can be contacted, messages that are moved back and forth between the service and its consumers, policies that the service adheres to, and consumers that interact with the service.

# SOA COMPONENTS

- **Services**
  - provide a distinct business function, as a coarse-grained piece of logic, implementing all of the functionality promised by the contracts it exposes
- **Contract (or API)**
  - collection of all the messages supported by the service is known as the service's contract
- **Endpoint**
  - universal resource identifier (URI), such as an address or a specific place, where the service can be found
  - a specific contract can be exposed at a specific endpoint
- **Message**
  - unit of communication in SOA
  - messages can come in many different forms (as HTTP requests, Simple Object Access Protocol (SOAP) messages, ...)
- **Service Consumer**
  - the software components that interact with a service via messaging.
  - can be either client applications or other services

# SOA COMPONENTS

- **Policy**

- just as an interface or contract separates specifications from implementations, *policies separate dynamic specifications from static or semantic specifications*
- defines the terms and conditions for making a service available for service consumers
- can be updated at runtime and they're externalized from the business logic.
  - a policy specifies dynamic properties, such as security (encryption, authentication, authorization), auditing, service-level agreements (SLAs), and so on

# SOA PRINCIPLES /1

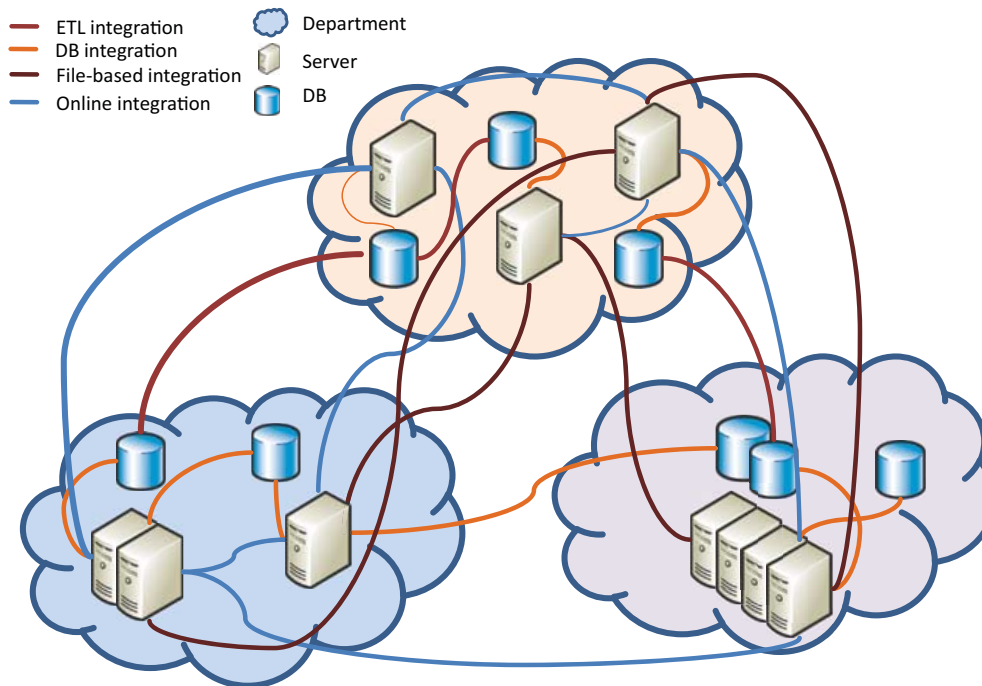
- Services are **reusable**
  - regardless of whether immediate reuse opportunities exist, services are designed to support potential reuse.
- Services share a **formal contract**
  - for services to interact, they need not share anything but a formal contract that describes each service and defines the terms of information exchange
- Services are **loosely coupled**
  - services must be designed to interact without the need for tight, cross-service dependencies.
- Services **abstract underlying logic**
  - the only part of a service that is visible to the outside world is what is exposed via the service contract.
  - the underlying logic, beyond what is expressed in the descriptions that comprise the contract, is invisible and irrelevant to service requestors.

# SOA PRINCIPLES /2

- Services are **composable**
  - services may compose other services. This allows logic to be represented at different levels of granularity and promotes reusability and the creation of abstraction layers
- Services are **autonomous**
  - the logic governed by a service resides within an explicit boundary. The service has control within this boundary and is not dependent on other services for it to execute its governance.
- Services are **stateless**
  - services should not be required to manage state information, as that can impede their ability to remain loosely coupled.
  - services should be designed to maximize statelessness even if that means deferring state management elsewhere.
- Services are **discoverable**
  - services should allow their descriptions to be discovered and understood by humans and service requestors that may be able to make use of their logic.

# SOA BENEFITS: AVOIDING “INTEGRATION SPAGHETTI”

- Generally speaking SOA **removes the dependency issues related to point-to point integration**
- The “Integration Spaghetti Problem”



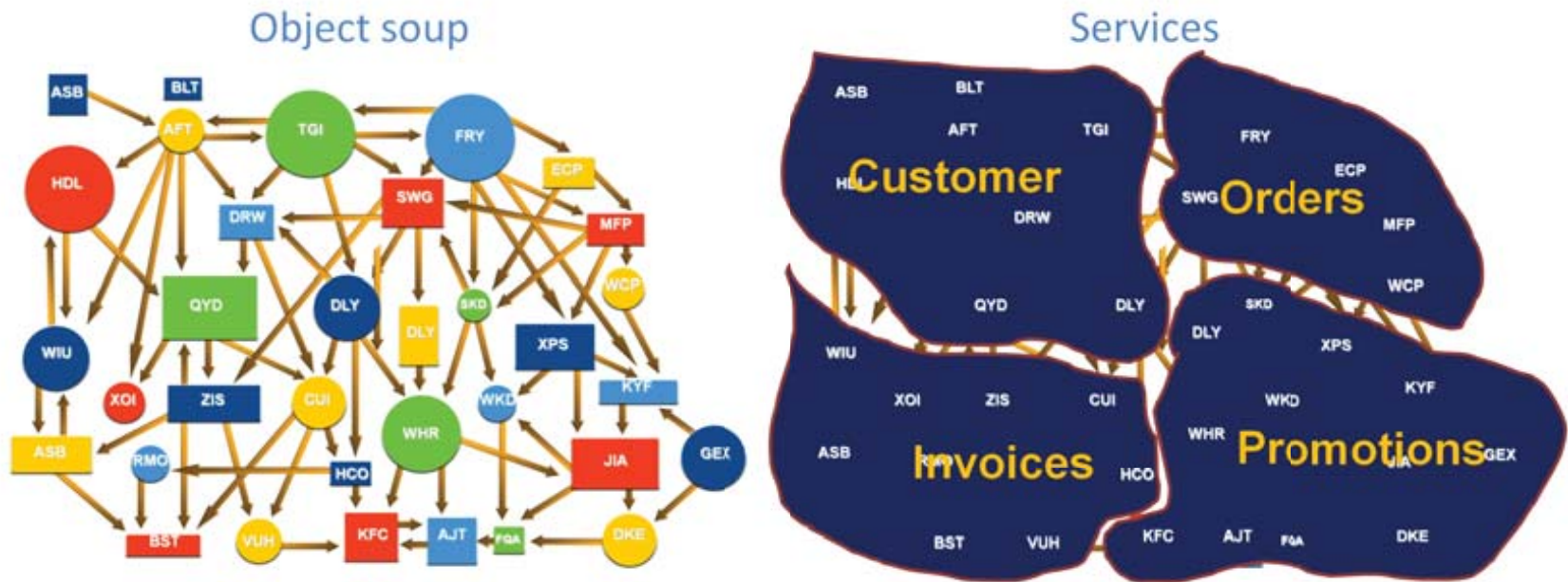
**Figure 1.2** Typical integration spaghetti in enterprise systems. Each department builds its own systems, and as people use the systems, they find they need information from other systems. Point-to-point integration emerges.

# SOA BENEFITS: AVOIDING “INTEGRATION SPAGHETTI”

- Removing dependency issues leads to improve:
  - **adaptability**
    - Isolating the internal structure of a service from the rest of the world lets you make changes more easily
    - the constraint is related to to adhere to the con tracts you publish.
  - **maintainability**
    - services can be maintained by dedicated, smaller teams and can be tested this way as well
  - **reusability**
    - this isn't reusability in the sense of “write once integrate any where,” but rather in the sense that you “don't throw everything out when you need different functionality.”



# SOA BENEFITS: AVOIDING THE “OBJECT SOUP”



**Figure 1.3** From object soup to well-formed services; one of the ideas behind SOA is to set explicit boundaries between larger chunks of logic, where each chunk represents a high-cohesion business area. This is an improvement on the more traditional approach, which more often than not results in an unintelligible object soup.

# SOA BENEFITS: BUSINESS LEVEL

**Table 1.2 SOA technical benefits and the business benefits they provide**

| SOA characteristic                               | Business benefit                                                                                                                                                             |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Easier maintenance and replacement of components | Easier replacement of existing business components<br>Better adaptability to accommodate changing business processes<br>Faster time to market for new business functionality |
| Standards-based service interfaces (contracts)   | Reduced effort to connect new systems<br>Easier partner integration<br>Enables automation of business process                                                                |
| Service autonomy                                 | Reduced downtime and lower operational costs                                                                                                                                 |
| Externalized policies                            | Ability to set service-level agreements<br>Easier integration                                                                                                                |

# SOA WITH WEB SERVICES

- A web service is an example of a SOA with a well-defined set of technology choices
  - based on **XML**, **SOAP** and **WSDL**
  - plus WS-\* standards stack (security, reliability, trust, ...)
- Main objective
  - providing seamless **interoperability** between heterogeneous middleware technology stacks and fostering the loose coupling of service consumer (requestor, client) and service provider
- Service model
  - service consumer and provider use **messages** to exchange invocation request and response information in the form of self-containing documents that make very few assumptions about the technological capabilities of the receiver.
  - there is no notion of a remote object reference that would require an object broker to manage a distributed memory address space
  - delivers interoperability for both the Remote Procedure Call (RPC) and messaging integration styles

# SOAP

- SOAP is an XML language defining a **message** architecture and message formats, hence providing a rudimentary processing protocol.
  - the SOAP document defines a top-level XML element called *envelope*, which contains a header and a body.
  - the SOAP *header* is an extensible container for message-layer infrastructure information that can be used for routing purposes (e.g., addressing) and Quality of Service (QoS) configuration (e.g., transactions, security, reliability).
  - the body contains the payload of the message.
- XML Schema is used to describe the structure of the SOAP message, so that SOAP engines at the two endpoints can marshal and unmarshal the message content and route it to the appropriate implementation.

# SOAP EXAMPLE

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Body>
  <m:GetPrice xmlns:m="http://www.w3schools.com/prices">
    <m:Item>Apples</m:Item>
  </m:GetPrice>
</soap:Body>
</soap:Envelope>
```

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Body>
  <m:GetPriceResponse xmlns:m="http://www.w3schools.com/prices">
    <m:Price>1.90</m:Price>
  </m:GetPriceResponse>
</soap:Body>
</soap:Envelope>
```

# WSDL

- An XML language for *defining interfaces* syntactically
  - the WSDL **port type** specifies multiple *abstract operations*
    - associated with some incoming and outgoing messages.
  - the WSDL **binding** links the set of abstract operations with concrete transport protocols and serialization formats
    - e.g. HTTP
- Service endpoints are addressed either on the transport level, i.e., with Universal Resource Identifiers (URIs) for SOAP/HTTP-bound endpoints, or on the messaging level via WS-Addressing.

# WSDL EXAMPLE

```
<message name="getTermRequest">
  <part name="term" type="xs:string"/>
</message>

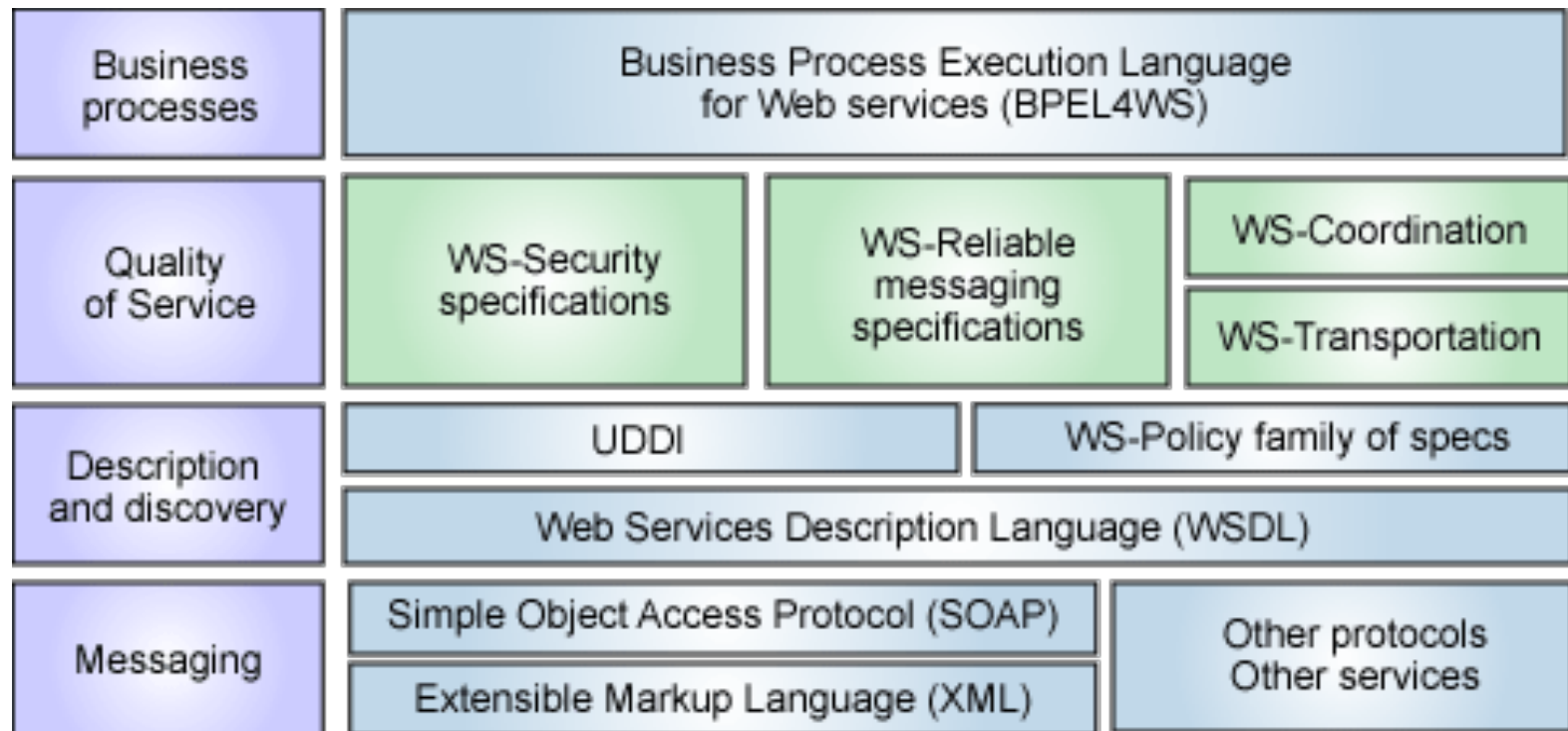
<message name="getTermResponse">
  <part name="value" type="xs:string"/>
</message>

<portType name="glossaryTerms">
  <operation name="getTerm">
    <input message="getTermRequest"/>
    <output message="getTermResponse"/>
  </operation>
</portType>

<binding type="glossaryTerms" name="b1">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http" />
  <operation>
    <soap:operation soapAction="http://example.com/getTerm"/>
    <input><soap:body use="literal"/></input>
    <output><soap:body use="literal"/></output>
  </operation>
</binding>
```

# THE FULL WS-\* STACK

- The WS-\* technology stack covers different QoS features required to ensure the interoperability of advanced middleware systems
  - given the modularity and composability of the approach, this has led to a fairly large set of WS-\* specifications





# IMPLEMENTATIONS

- Several implementations available
  - examples
    - .NET WCF
    - JAX-WS spec
      - implemented by various vendors
- Apache Axis2
  - <http://axis.apache.org/axis2/>
- Eclipse Web Tools platform
  - <http://www.eclipse.org/webtools/>

# THE RISING OF A REST-BASED SOA

- **REST**

- REpresentational State Transfer (REST) was originally introduced by Roy Fielding in his PhD as an *architectural style for building large-scale distributed hypermedia systems*
- Architectural style whose principles have been used to explain the excellent scalability of the HTTP 1.0 protocol and have also constrained the design of its following version, HTTP 1.1.
  - thus, the term REST very often is used in conjunction with HTTP.

# REST AS A SW ARCHITECTURE

- Overall, REST can be described as a technology and platform-independent architecture where ***loosely coupled components communicate via interfaces over standard web protocols.***
  - software, hardware, and data-centric designs maximize system efficiency, scalability, and network throughput.
  - the underlying principal is *simplicity*
- REST differs from other software architecture in that it marries the concepts common to software architecture with those of network architecture (portability, bandwidth management, throughput measurement, protocol latencies, and so on)
  - this combination makes REST ideal for distributed software systems where ***scalability***, in terms of both processing power and communication efficiency, are critical.

# RESTful WEB SERVICES

- RESTful web services
  - Web services based on REST principle for implementing remote procedure calls across the Web
  - gaining increased attention not only because of their usage in the Application Programming Interface (API) of many Web 2.0 services, but also because of the presumed simplicity of publishing and consuming a RESTful Web service
- *Nowadays RESTful web services and RESTful APIs are the de facto standard in consumer/web scale enterprises*
  - e.g. Twitter, Google

# SOA WITH REST

- **Service**
  - a software program that makes its functionalities available via published technical interface, called service contract
- **Service Contract and Capabilities**
  - service contract broken down in terms of service *capabilities*
    - each expressing a function offered by the service to the other service programs
- **Service Consumer**
  - runtime role assumed by the software program when it accesses and invokes a service or more specifically when it sends a message to a service capability
  - upon receiving the request, the service begins processing
  - it may or may not return a corresponding message to the service consumer

# UNIFORM CONTRACT ELEMENTS

- **Resource identifier syntax**
  - *how we can express where the data is being transferred from/to*
  - HTTP URI
- **Methods**
  - *what are the protocol mechanisms used to transfer the data*
  - HTTP methods
    - POST to create a resource
    - GET to retrieve (a representation of) the state of a resource
    - PUT to change the state of a resource or to update it
    - DELETE to remove or delete a resource
- **Media types**
  - *what types of data is being transferred*
  - XML, JavaScript Object Notation (JSON)

# RESOURCE IDENTIFIER SYNTAX

- Resource identifier syntax: URI

**{scheme} : // {authority} {path} ? {query} # {fragment}**

- the (optional) query contains a set of parameters that defines the search criteria used by the server to find a set of matching resources.
- Examples
  - http://customer.example.com/customer/C081
    - authority/service: http://customer.example.com
    - resource id: customer/C081
  - http://customer.example.com/invoice?total-less-than=100USD#page2
  - http://customer.example.com/ValidateTimesheet/start

# SERVICE MODELS

- Entity Services
- Task Services
- Utility Services



# ENTITY SERVICES

- Services establishing a functional boundary associated with one or more related ***business entities***
  - e.g. invoice, claim, customer,...
- Example: Invoice service

## Capabilities

- GET /invoice/{invoice-id}
- PUT /invoice/{invoice-id}/customer
- PUT /invoice/{invoice-id}/date
- DELETE /invoice/{invoice-id}

# TASK SERVICES

- Services establishing a functional boundary associated with one or more related ***business tasks***
  - the name typically contains a verb
  - typically few service capabilities, to control the task
- Example: `ValidateTimesheet` service

## Capabilities

- `POST /start/{timesheet,request-id}`
- `GET /task/{id}`
- `DELETE /task/{id}`

# UTILITY SERVICES

- Services grouping related services capabilities that may have large functional boundaries
  - e.g. wrapping legacy system
- Example: HRSystem service
  - GET /timesheet/{timesheet-id}
  - GET /employee/{employee-id}/
  - GET /employeehistory/{date}/
  - GET /payroll/{scale-id}

# IMPLEMENTATION & TECHNOLOGIES

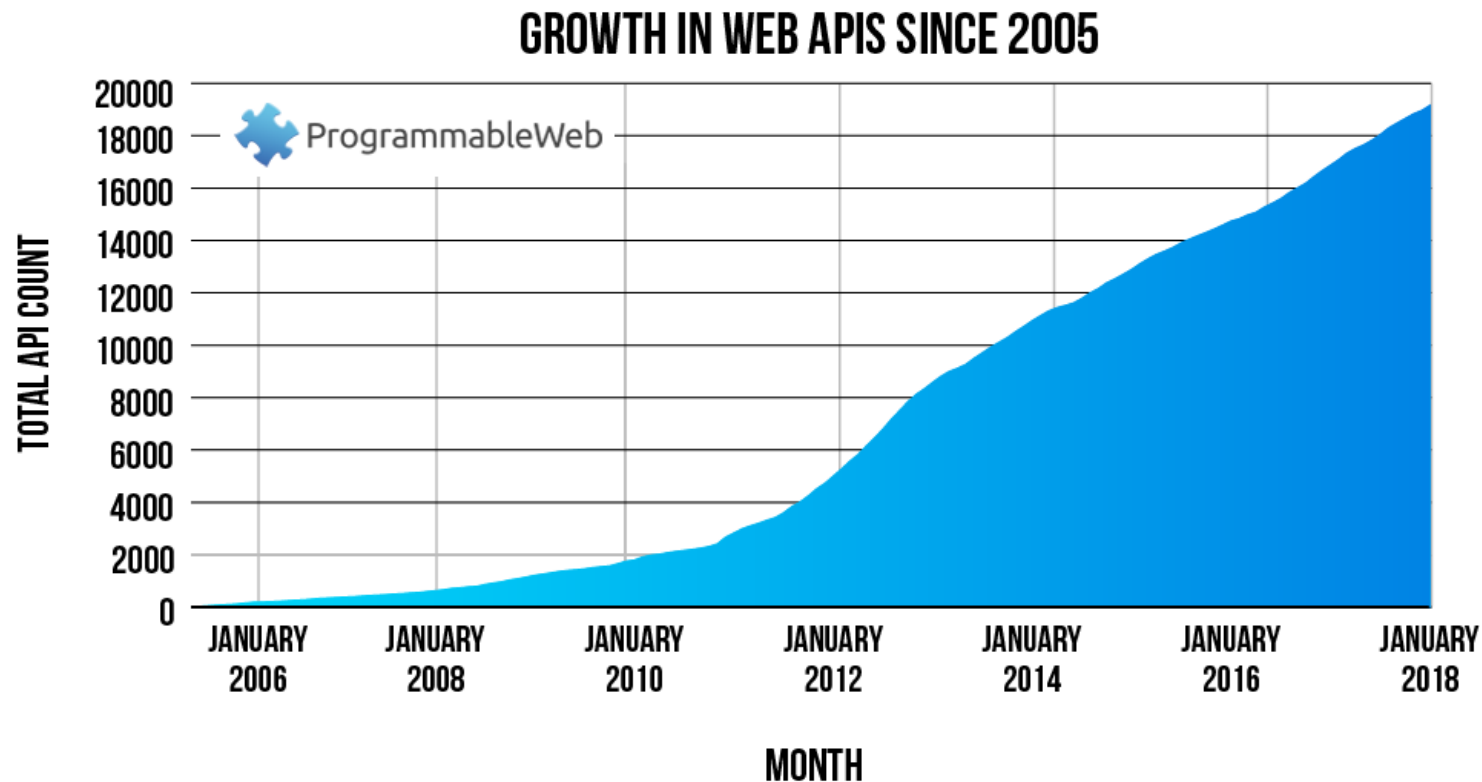
- Any HTTP API (client/server) would be enough, but specific language API to make it seamless the usage in the hosting language
- Example: JAX-RS: Java API for RESTful Web Services
  - Java programming language API that provides support in creating web services according to the Representational State Transfer (REST) architectural pattern
  - Implementations (many)
    - Spring
    - Vert.x
    - ...

# SPECIFYING REST API

- **Open API** as an interface definition language (IDL) for REST API
  - <https://www.openapis.org/>
  - evolved from the **Swagger** open source project
- The Swagger project (<http://swagger.io/>)
  - set of tools for developing and documenting REST APIs
  - It includes tools that generate client stubs and server skeletons from an interface definition.
- Other examples
  - **RAML** (RESTful API Modeling Language)
    - “The simplest way to design APIs”
    - <http://raml.org/>
  - **RESTlet**
    - “The most complete platform for API developers”
    - <https://restlet.com/>
  - ...

# A WORLD OF (WEB) API

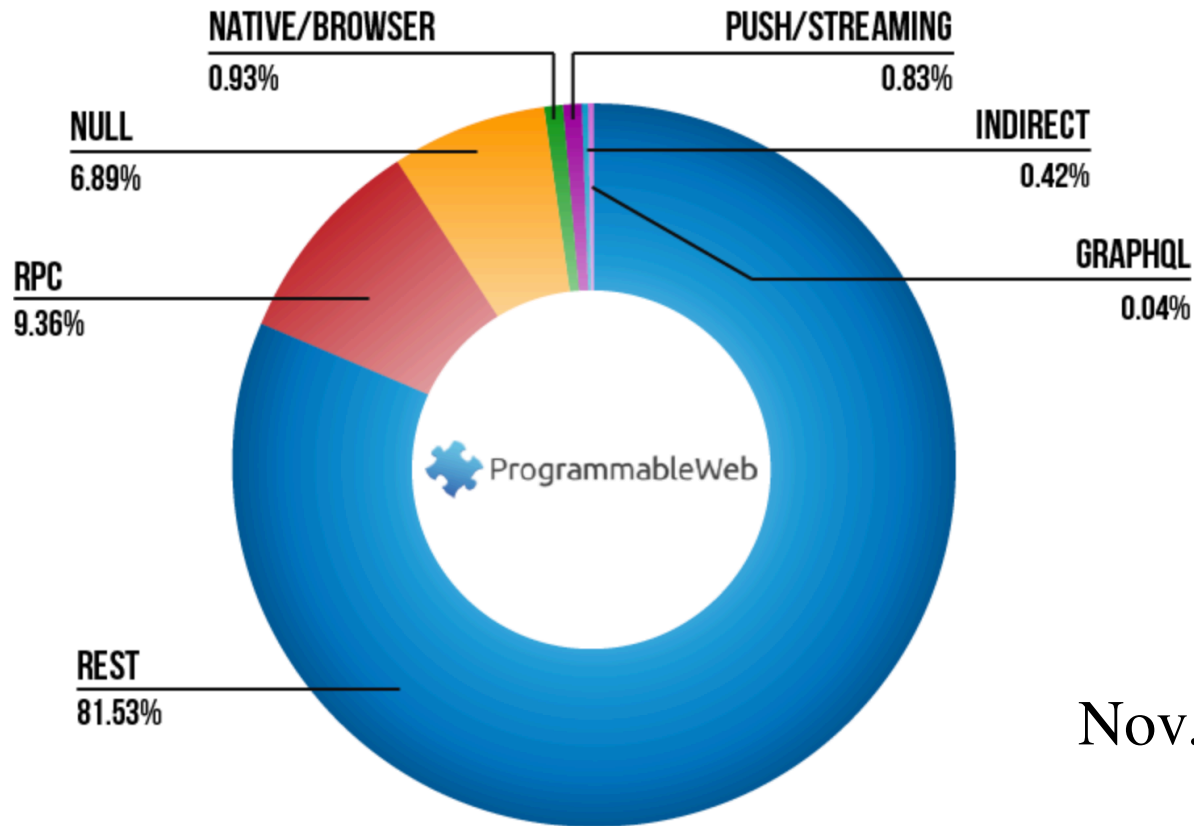
- The rapid growth of programmatic interfaces for Web applications has recently revolutionized online content integration and created new opportunities for vendors to build developer ecosystems



(graph from “ProgrammableWeb” web site, no more available)

# API ARCH STYLES ADOPTION

## ARCHITECTURAL STYLES RECORDED IN PROGRAMMABLEWEB DIRECTORY



Nov. 2017

Figure 2: The percentages of API Architectural Styles for profiles in the ProgrammableWeb API directory

(graph from “ProgrammableWeb” web site, no more available)

# API QUERY LANGUAGES

- Approaches to give clients the possibility to describe flexible queries, making it easier to evolve APIs over time
  - **GraphQL**
    - <https://graphql.org/>



# PROGRAMMATIC WEB INTERFACE PERVASIVENESS

- Today, programmatic Web interfaces have become a core feature that developers expect from any Web application.
  - services let application vendors track who is using applications, as well as how and why, allowing them to more effectively foster affiliates and drive traffic back to applications.
  - by offering services, vendors provide application functionality and start charging for it.
  - a small fee per service call is acceptable for many subscribers, while big opportunities exist for profiting from usage volume

# FROM SERVICES TO MICROSERVICES

# MICROSERVICES

- **Architectural style** to develop *a single application as a suite of small services*
  - each running in **its own process**
  - and communicating with lightweight mechanisms
    - REST API or lightweight messaging
- These services are built around *business capabilities* and *independently deployable* by fully automated deployment machinery
- There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.

# SOFTWARE ARCHITECTURES

- *"The software architecture of a computing system is the set of **structures** needed to reason about the system, which comprise **software elements**, **relations among them**, and **properties of both**."*
  - Documenting Software Architectures by Bass et al.
- Key points: Decomposition and modularity
  - decomposition facilitates the division of labor and knowledge.
    - it enables multiple people (or multiple teams) with possibly specialized knowledge to work productively together on an application.
  - It defines how the software elements interact.
- Architecture is important because it enables an application to satisfy *quality of service* requirements

# ARCHITECTURAL STYLES

- *An architectural style, then, defines a family of such systems in terms of a **pattern of structural organization**. More specifically, an architectural style determines the vocabulary of **components** and **connectors** that can be used in instances of that style, together with a set of **constraints** on how they can be combined.*
  - David Garlan and Mary Shaw. An Introduction to Software Architecture, January 1994
- Main examples
  - **layered**
  - **hexagonal**
  - **microservices**

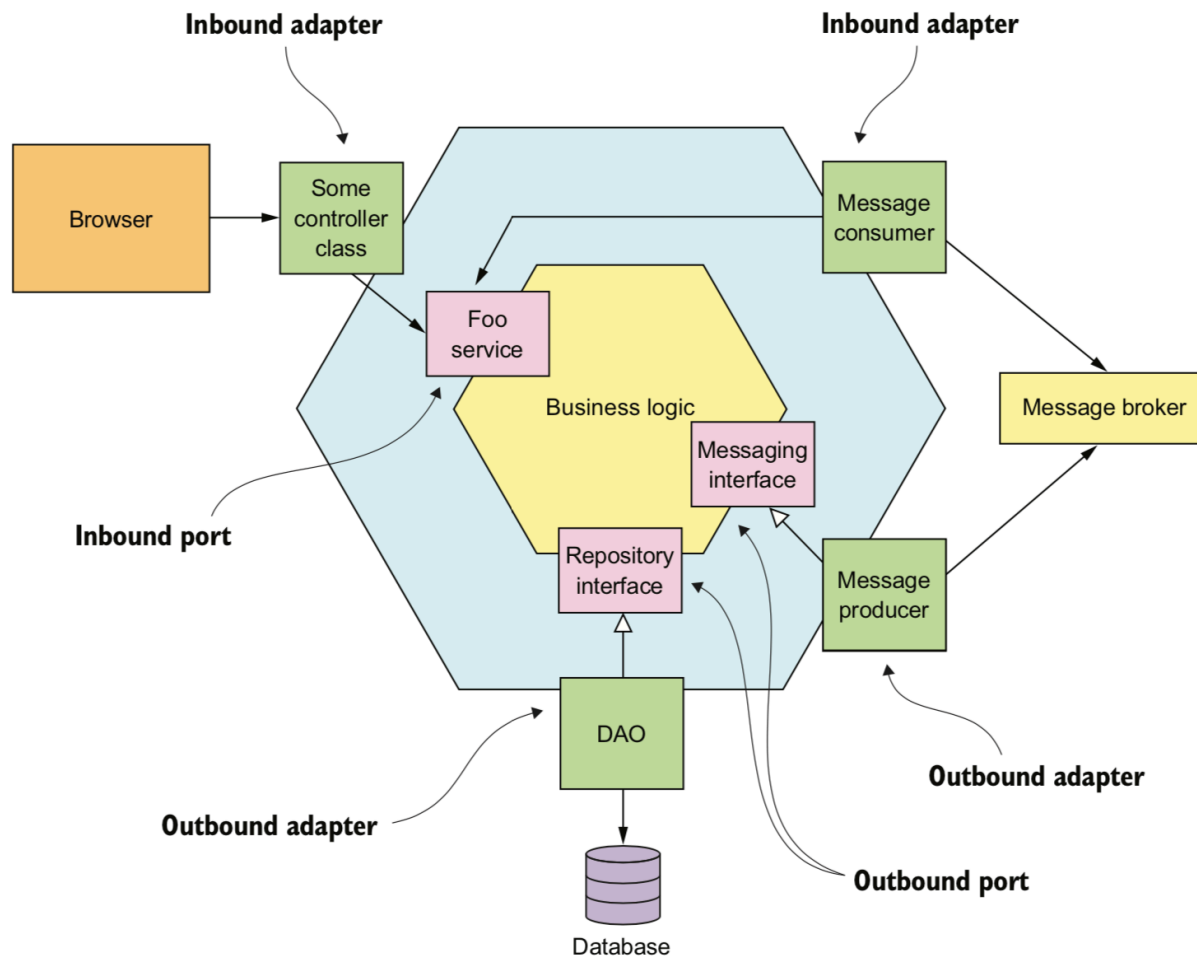
# LAYERED ARCHITECTURAL STYLE

- Organizes software elements into layers
  - each layer has a well-defined set of *responsibilities*
  - constrained dependencies between the layers.
    - a layer can only depend on either the layer immediately below it (if strict layering) or any of the layers below it.
- Example: 3-Layer architecture:
  - *Presentation layer*—Contains code that implements the user interface or external APIs
  - *Business logic layer*—Contains the business logic
  - *Persistence layer*—Implements the logic of interacting with the database
- Drawbacks
  - Single presentation layer—It doesn't represent the fact that an application is likely to be invoked by more than just a single system.
  - Single persistence layer—It doesn't represent the fact that an application is likely to interact with more than just a single database.
  - Defines the business logic layer as depending on the persistence layer—In theory, this dependency prevents you from testing the business logic without the database.

# HEXAGONAL ARCHITECTURE

- Organizes the logical view in a way that *places the business logic at the center*
  - the application has one or more *inbound adapters* that handle requests from the outside by invoking the business logic.
  - the application has one or more *outbound adapters* that are invoked by the business logic and invoke external applications.
  - the business logic doesn't depend on the adapters. Instead, they depend upon it.
- The business logic has one or more ports.
  - A **port** defines a set of operations and is how the business logic interacts with what's outside of it.
    - Java example: a Java interface.
  - Two kinds of ports: inbound and outbound ports.
    - an *inbound port* is an *API* exposed by the business logic, which enables it to be invoked by external applications.
      - e.g. a service interface, defining a service's public methods.
    - an *outbound port* is *how the business logic invokes external systems*.

# HEXAGONAL ARCHITECTURE



An important benefit of the hexagonal architectural style is that it decouples the business logic from the presentation and data access logic in the adapters. The business logic doesn't depend on either the presentation logic or the data access logic.

Figure 2.2 An example of a hexagonal architecture, which consists of the business logic and one or more adapters that communicate with external systems. The business logic has one or more ports. Inbound adapters, which handled requests from external systems, invoke an inbound port. An outbound adapter implements an outbound port, and invokes an external system.

(Chris Richardson. *Microservices Patterns*. Manning, 2019)

Service-Oriented Architectures

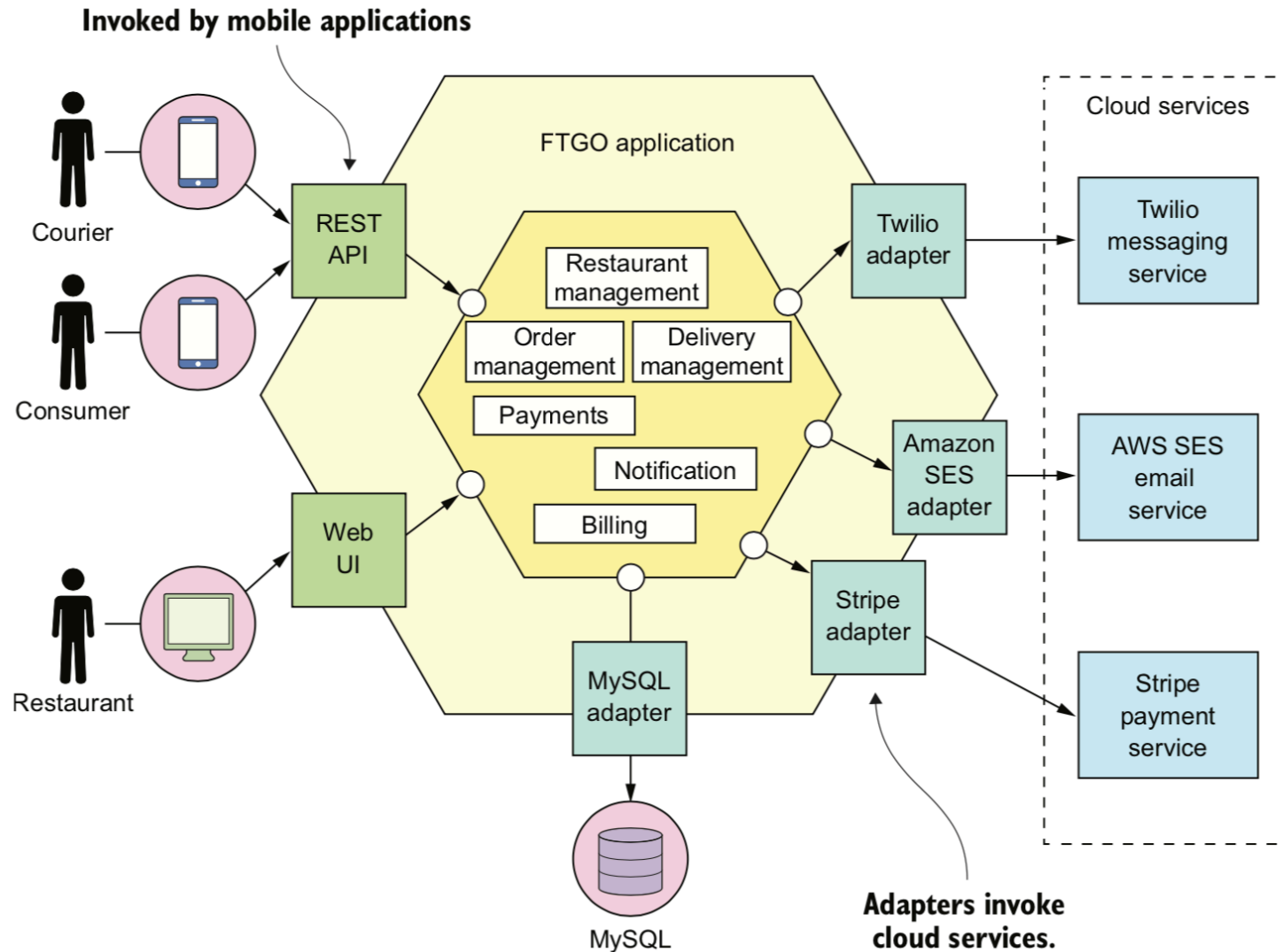


# MONOLITHIC ARCHITECTURE

- Architectural style that *structures the implementation view as a single component: a single executable or WAR file*
  - this definition says nothing about the other views: a monolithic application can, for example, have a logical view that's organized along the lines of a hexagonal architecture.
- Pattern: monolithic architecture
  - <https://microservices.io/patterns/monolithic.html>

# HEXAGONAL/MONOLITHIC ARCHITECTURE

FTGO = “Food to go”  
Application Example used in Richardson’s book



(Chris Richardson. *Microservices Patterns*. Manning, 2019)

# MICROSERVICE ARCHITECTURE

- Architectural style that structures the *implementation view* as a set of *multiple components*: executables or WAR files
  - the components are services, and the connectors are the communication protocols that enable those services to collaborate
  - each service has its own logical view architecture, which is typically a hexagonal architecture
- Pattern: microservice architecture
  - <http://microservices.io/patterns/microservices.html>

# MICROSERVICE ARCHITECTURE

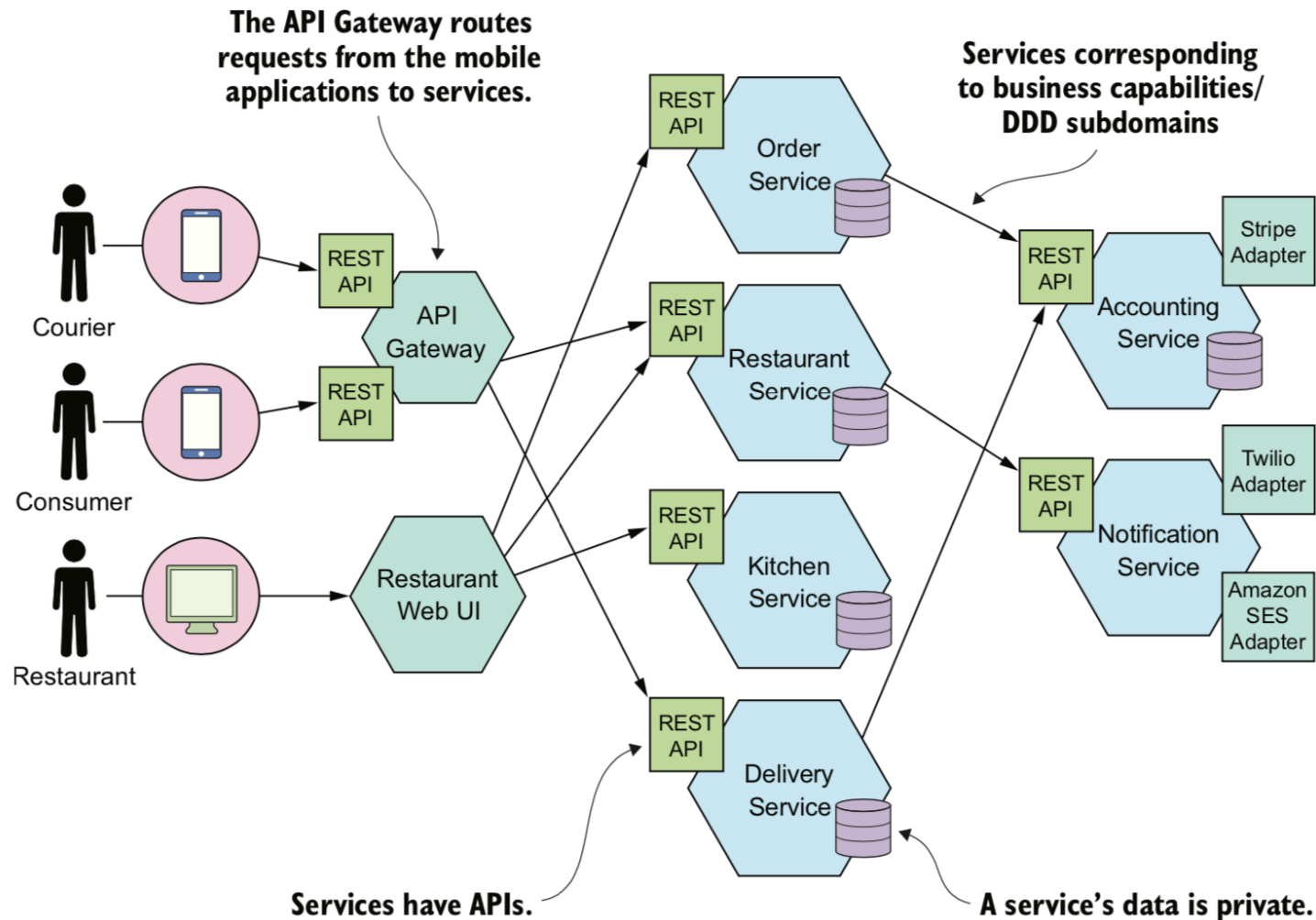
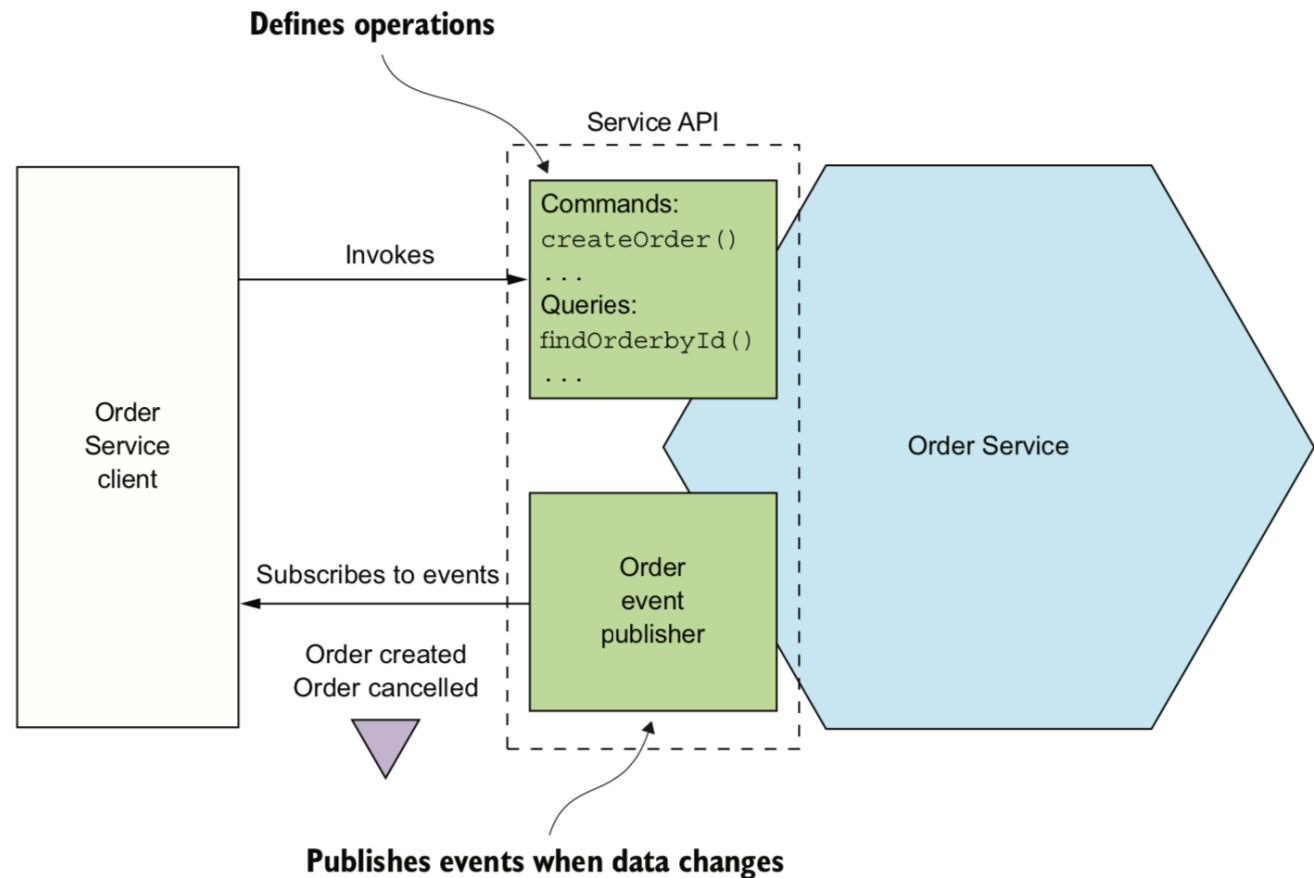


Figure 2.3 A possible microservice architecture for the FTGO application. It consists of numerous services.

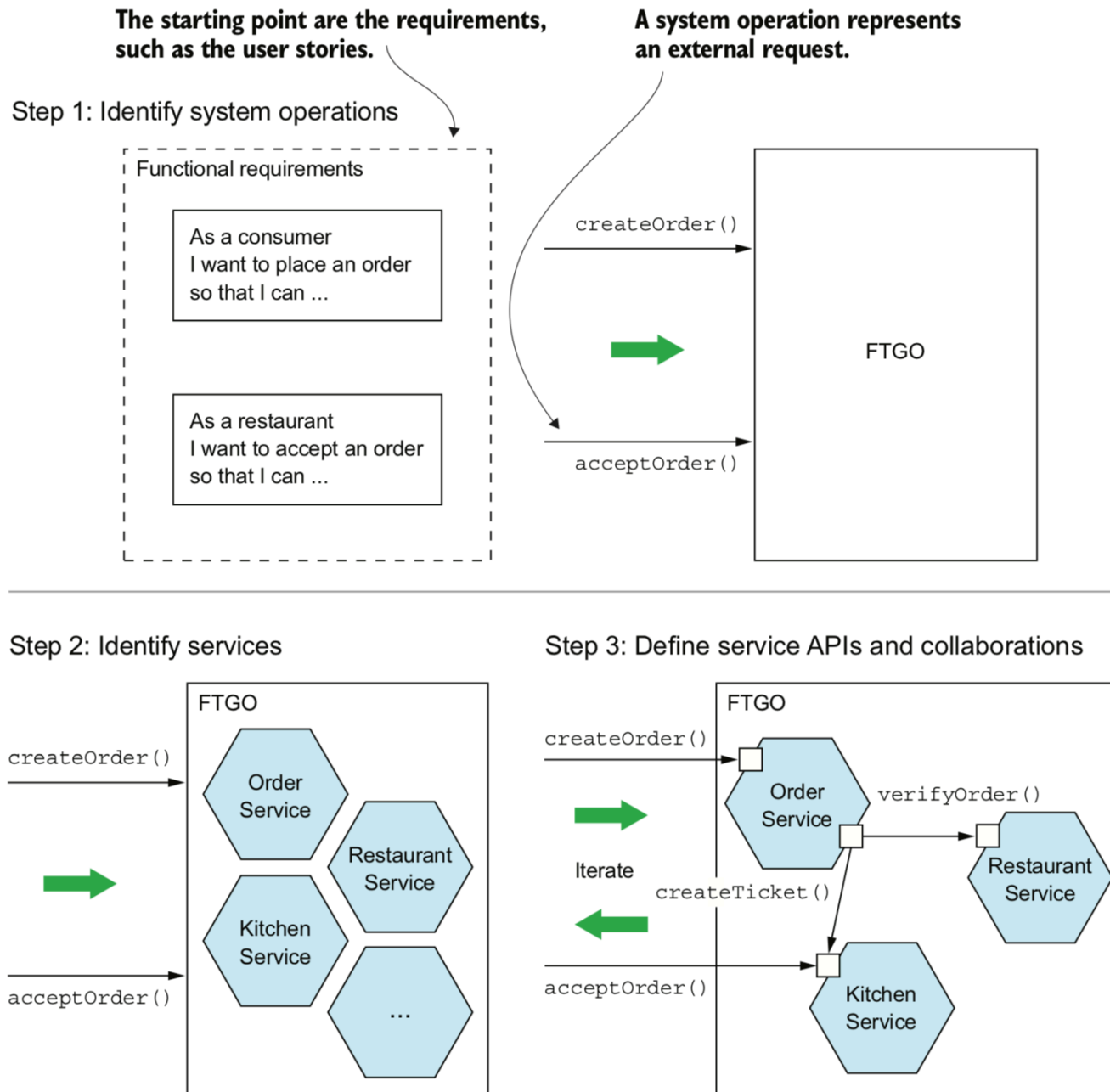
# SERVICE API

- Commands
- Query
- Events



**Figure 2.4** A service has an API that encapsulates the implementation. The API defines operations, which are invoked by clients. There are two types of operations: commands update data, and queries retrieve data. When its data changes, a service publishes events that clients can subscribe to.

Screenshot



**Figure 2.5** A three-step process for defining an application's microservice architecture

# LOOSE COUPLING

- Services are loosely coupled
  - all interaction with a service happens via its **API**, which encapsulates its implementation details
  - the implementation of the service can change without impacting its clients
  - key to improving an application's development time attributes, including its maintainability and testability

# INTERACTION STYLES

- One-to-one vs. one-to-many
  - **one-to-one**
    - Each client request is processed by exactly one service
  - **one-to-many**
    - Each request is processed by multiple services.
- Synchronous vs. asynchronous
  - **synchronous**
    - the client expects a timely response from the service and might even block while it waits.
  - **asynchronous**
    - the client doesn't block, and the response, if any, isn't necessarily sent immediately.

|              | one-to-one                                             | one-to-many                                  |
|--------------|--------------------------------------------------------|----------------------------------------------|
| Synchronous  | Request/response                                       | —                                            |
| Asynchronous | Asynchronous request/response<br>One-way notifications | Publish/subscribe<br>Publish/async responses |



# ONE-TO-ONE MAIN EXAMPLES

- Main examples of one-to-one interactions:
  - **request/response**
    - A service client makes a request to a service and waits for a response. The client expects the response to arrive in a timely fashion. It might even block while waiting.
    - This is an interaction style that generally results in services being tightly coupled.
  - **asynchronous request/response**
    - A service client sends a request to a service, which replies asynchronously
    - The client doesn't block while waiting, because the service might not send the response for a long time.
  - **one-way notifications**
    - A service client sends a request to a service, but no reply is expected or sent

# ONE-TO-MANY MAIN EXAMPLES

- **Publish/subscribe**
  - a client publishes a notification message, which is consumed by zero or more interested services.
- **Publish/async responses**
  - A client publishes a request message and then waits for a certain amount of time for responses from interested services.

# MAIN IPC TECHNOLOGIES for API

- **REST**
  - used as transport for RPC (Richardson's book) (=> coupling)
  - used not for RPC, but REST-ful style (=> uncoupling)
    - + websocket for event-oriented interactions
- **Messages & MOM**
  - asynchronous (=> uncoupling)
- Recent entries
  - **gRPC**
    - *"A high-performance universal RPC framework"*
    - <https://grpc.io>
  - **GraphQL**
    - *"a query language for your API"*
    - <https://graphql.org>

# MICROSERVICES ORGANIZATION

- Building applications as suites of services
  - services are *independently deployable* and *scalable*
  - encapsulation typically concerns *business* functions
  - each service also provides a *firm module boundary*
    - even allowing for different services to be written in different programming languages
  - they can also be managed by different teams

# SERVICE-BASED COMPONENTISATION

- Component-oriented software [90ies, 00ies]
  - build systems by *plugging together components*, much in the way we see things are made in the physical world.
  - components as a unit of software that is independently replaceable and upgradeable
- Micro services architectures => **components = services**
  - services as *out-of-process components* who communicate with a mechanism such as a web service request, or remote procedure call.
    - vs. library in which components that are linked into a program and called using *in-memory function calls*

# BUSINESS-ORIENTED DECOMPOSITION

- Business-oriented vs. technological oriented decomposition of (large) applications
  - monolith approach => more technological-oriented decomposition
    - separation between UI, server-side logic, database..
      - => UI teams, server-side logic teams, and database teams.
    - when teams are separated along these lines, even simple changes can lead to a cross-team project taking time and budgetary approval.
  - microservices approach => splitting up into services ***organized around business capability***
    - services taking a broad-stack implementation of software for that business area, including user-interface, persistent storage, and any external collaborations
    - teams are *cross-functional*, including the full range of skills required for the development: user-experience, database, and project management

# DOMAIN-DRIVEN DESIGN

- Business-oriented decomposition => **Domain-driven design** (DDD) [\*]
  - DDD is an approach for building complex software applications centered on the development of an object-oriented *domain model*
  - A domain model captures knowledge about a domain in a form that can be used to solve problems within that domain.
    - it defines the vocabulary used by the team (the Ubiquitous Language)
    - the domain model is closely mirrored in the design and implementation of the application
- Two main concepts in DDD useful when applying the microservice architecture: **subdomains** and **bounded contexts**
  - *Subdomain* - DDD decomposes the domain in separate subdomains (and subdomain models), identified using the same approach as identifying business capabilities
  - *Bounded Contexts* - DDD calls the scope of a domain model a bounded context => when using the microservice architecture, *each bounded context is a service or possibly a set of services.*

(\*) Eric Evans. *Domain-Driven Design*. Addison Wesley, 2003

# “API-FIRST” APPROACH

- API-first approach to defining services (\*)
  - First you write the *interface definition*
    - using an IDL (Interface Definition Language)
    - ref: Open API 3.0, RAML, etc.
  - then you review the interface definition with the client developers.
  - only after iterating on the API definition do you then implement the service
  - doing this up-front design increases your chances of building a service that meets the needs of its clients.

(\*) [www .programmableweb.com/news/how-to-design-great-apis-api-first-design-and-raml/how-to/](http://www.programmableweb.com/news/how-to-design-great-apis-api-first-design-and-raml/how-to/) 2015/07/10



# SERVICES AS COMPONENTS

- *Services are independently deployable*
  - if you have an application that consists of a multiple libraries in a single process, a change to any single component results in having to redeploy the entire application
  - if that application is decomposed into multiple services, you can expect many single service changes to *only require that service to be redeployed*
    - minimizing changes to service interfaces resulting through cohesive service boundaries and evolution mechanisms in the service contracts

# SERVICES AS COMPONENTS

- *More explicit component interface*
  - most languages do not have a good mechanism for defining an explicit published Interface
    - often it's only documentation and discipline that prevents clients breaking a component's encapsulation, leading to overly-tight coupling between components.
  - services make it easier to avoid this by using explicit remote call mechanisms
- **Important downsides: Remote calls vs. In-process calls**
  - remote calls are more expensive than in-process calls, and thus remote APIs need to be coarser-grained, which is often more awkward to use

# THE CHALLENGES OF MICROSERVICES

- Require a distributed system design
- Network latency
- Interprocess Communication complexity
  - synchronous IPC reduce availability
- Maintaining data consistency across services
- Obtaining a consistent view of data

# PROS & CONS OF MICROSERVICES

## Microservices provide **benefits**...

- **Strong Module Boundaries:** Microservices reinforce modular structure, which is particularly important for larger teams.



- **Independent Deployment:** Simple services are easier to deploy, and since they are autonomous, are less likely to cause system failures when they go wrong.



- **Technology Diversity:** With microservices you can mix multiple languages, development frameworks and data-storage technologies.

## ...but come with costs

- **Distribution:** Distributed systems are harder to program, since remote calls are slow and are always at risk of failure.



- **Eventual Consistency:** Maintaining strong consistency is extremely difficult for a distributed system, which means everyone has to manage eventual consistency.



- **Operational Complexity:** You need a mature operations team to manage lots of services, which are being redeployed regularly.

# FROM SERVICES TO X-AS-A-SERVICE - CLOUD COMPUTING

# CLOUD COMPUTING

- Cloud computing is nowadays one of the main choices to design distributed large-scale applications, featuring
  - **scalability**
  - **openness & heterogeneity**
  - **availability**
- This module: introduction about Cloud PAAS and Cloud app development using PAAS
  - cloud apps can be developed on top of available Platform-as-a-Service infrastructures
  - this calls for adopting specific principles to design and develop applications

# CLOUD COMPUTING

- A paradigm shift in the delivery architecture of information services.
  - moving services, computation and/or data—for cost and business advantage—*off-site to an internal or external, location-transparent, centralized facility or contractor*
    - **off-premise** (cloud) vs. on-premise (classic)
  - drivers: Google, Yahoo, Amazon, and others have built large, purpose-built architectures to support their applications
    - ...and taught the rest of the world how to do massively scalable architectures to support compute, storage, and application services
- Outcome
  - by making data available in the cloud, it can be more easily and ubiquitously accessed, often at much lower cost, increasing its value by enabling opportunities for enhanced collaboration, integration, and analysis on a shared common platform
  - ...but there are also performance and security concerns to check

# MOVING CAP-EX TO OP-EX

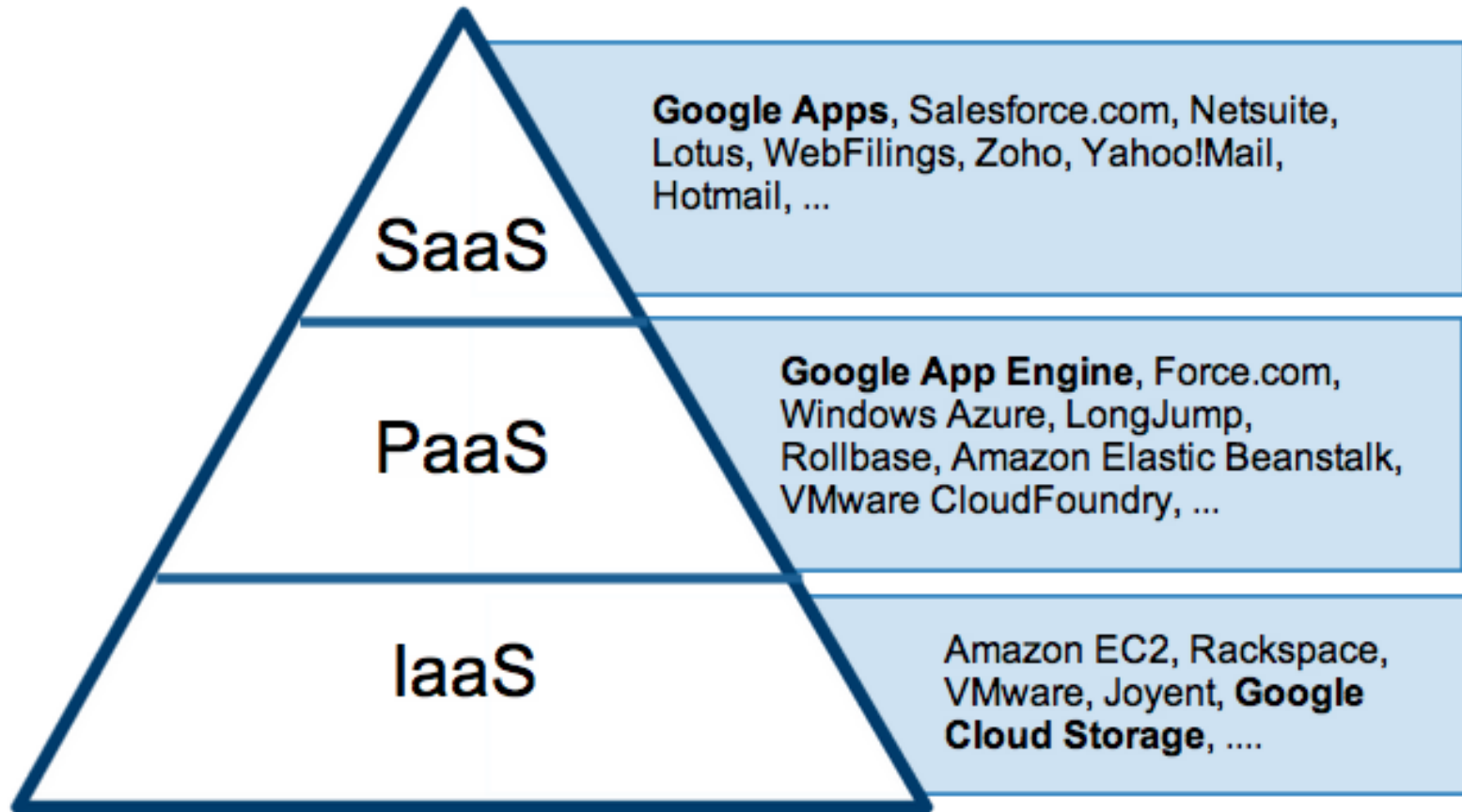
- In the past, developing an application service required a large *CapEx* (capital expense) to build infrastructure for peak service demand before deployment.
- Cloud computing addresses this problem by *allowing expenses to track closely with resource use*, thus following income rather than having to purchase for peak capacity before income is realized.
- Running application services on a cloud platform accomplishes this in three fundamental ways:
  - ***it moves CapEx to OpEx*** (operational expense), closely correlating expenses with resource use.
  - It allows service owners to eliminate significant system-administration head count by avoiding the need for internally purchased servers.
  - It smooths the path to service scaling by not requiring the CapEx-intensive architectural changes needed to scale up service capacity in the event of service success.
- Because the cost of deploying new services is much lower and expenses track real usage, businesses can develop and deploy more services without fear of writing off huge capital investments for dedicated infrastructure that may never be needed



# SERVICE MODELS

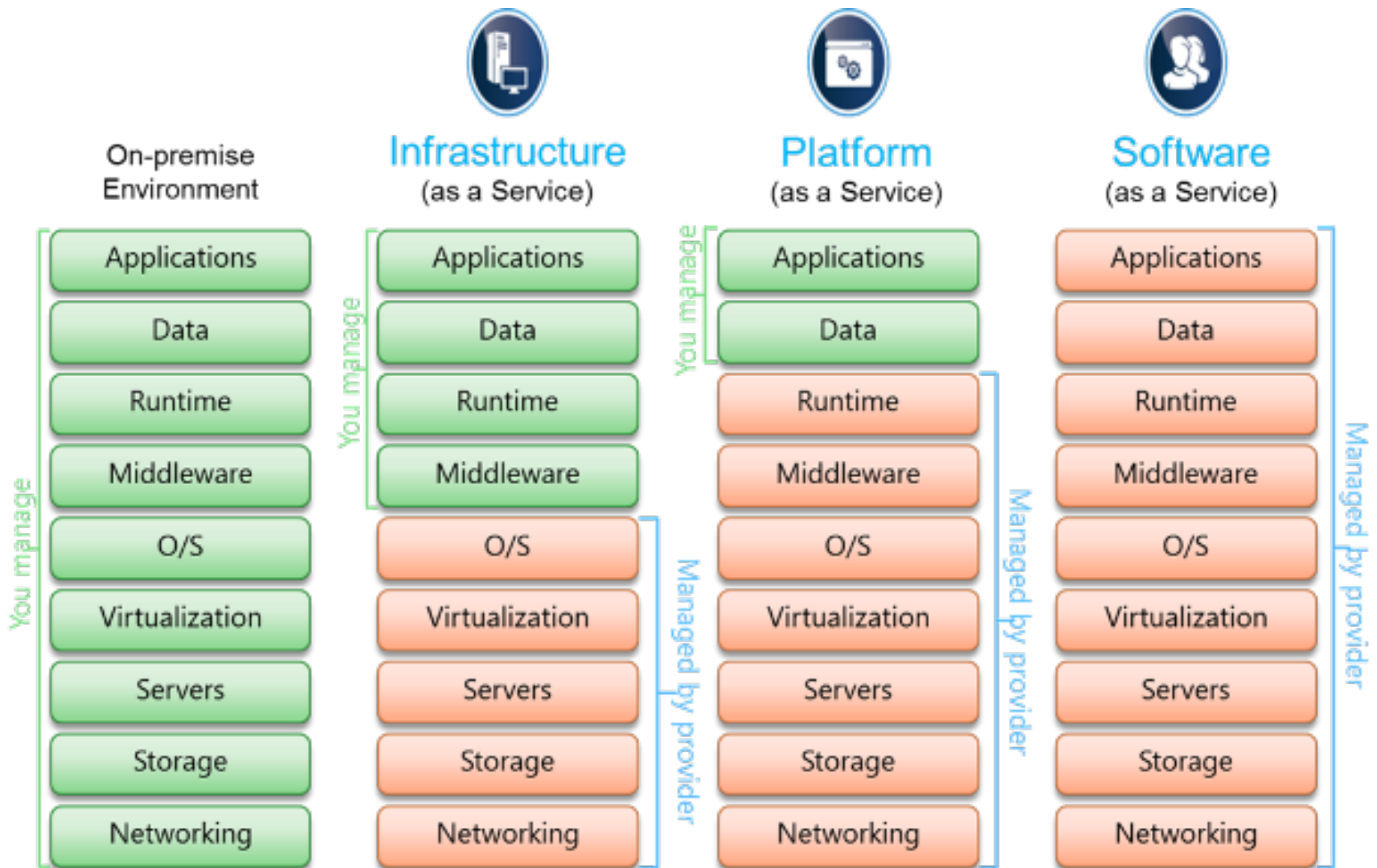
- Cloud computing can be divided into three main areas
  - **SaaS (software-as-a-service)**
    - WAN-enabled application services
    - e.g., Google Apps, [salesforce.com](https://salesforce.com), WebEx
  - **PaaS (platform-as-a-service)**
    - foundational elements to develop new applications
    - e.g., Google Application Engine, Windows Azure,...
  - **IaaS (infrastructure-as-a-service)**
    - providing computational and storage infrastructure in a centralized, location-transparent service
    - e.g., Amazon EC2, Google Computing Engine, Open Stack

# SERVICE MODELS



Source: Gartner AADI Summit Dec 2009

# SERVICE MODELS & LAYERS



# (SOME) MAIN PLAYERS

- Amazon AWS Cloud
  - <https://aws.amazon.com/>
- Google Cloud Platform
  - <https://cloud.google.com/>
- Windows Azure Platform
  - <https://azure.microsoft.com/>
- Heroku
  - <https://www.heroku.com/home>
- Open Source
  - Cloud Foundry (PAAS)
    - <https://www.cloudfoundry.org/>
  - ...
- ...

# COMMONALITIES AMONG AVAILABLE PAAS

- SOA
  - based on REST (& GraphQL, gRPC, ...)
- Set of available core services (API) provided by the platform
  - to store/retrieve data
  - to execute/manage long-term tasks
  - communication
  - app configuration and management

# FROM PAAS TO FAAS

- **FAAS** = Function-as-a-Service
  - Moving function-oriented, serverless code to the cloud
    - triggered by developer-defined events
- Similar to PAAS, but more lightweight
  - more reactive, more easy to scale (auto)
- Examples
  - Amazon Lambda
  - Google Cloud

# SERVERLESS COMPUTING

- FAAS is nowadays referred as **serverless computing** [\*]
  - No servers / operating systems to configure or manage
  - Pay-per-invocation billing model
  - Ability to automatically scale with usage
  - Built-in availability and fault tolerance

[\*] From book: *Serverless Design Patterns and Best Practices* - B. Zambrano

# WHEN TO USE SERVERLESS

- When to use *serverless*
  - well suited to problems that are or can be broken into *stateless* and *computationally small and predictable* subsystems
  - serverless functions are ephemeral
    - have a known lifetime
      - temporal constraints imposed by the platforms
  - application state may exist => stored into data store
    - functions themselves retain no state between invocation

[\*] From book: *Serverless Design Patterns and Best Practices* - B. Zambrano



# BEYOND THIS OVERVIEW

# ENGINEERING LARGE-SCALE SERVICE-ORIENTED SYSTEMS

- **Designing Large-Scale Software Systems & Infrastructures**
  - **Software Architectures & Patterns & Technologies**
    - Microservices Patterns
      - Event sourcing, Circuit Breaker, CRQS, API gateway, Saga, ...
    - Event-Driven Architectures and Event-Driven Microservices
  - Cloud computing & Data-Intensive Distributed Applications
- **Methodologies & Processes**
  - Domain-Driven Design & DevOps

# ENGINEERING SITUATED SERVICE-ORIENTED SYSTEMS

- Architectures for **Internet of Things** & Web of Things systems & applications
  - Large-scale + interaction/modeling with the physical world/reality
  - **Digital Twin** architectural pattern & paradigm

# ENGINEERING SITUATED AND AUTONOMOUS DISTRIBUTED SYSTEMS

- Models, architectures, technologies for integrating AI in the design/engineering of smart distributed applications and Internet-based intelligent autonomous systems
  - **Autonomous Agents** and **Multi-Agent Systems**

# REFERENCES

- Thomas Erl, *Service-Oriented Architecture Concept, Technology, and Design*. Prentice-Hall
- Tomas Vitvar, Steve Vinoski, and Cesare Pautasso. 2012. *Programmatic Interfaces for Web Applications*. IEEE Internet Computing 16, 4 (July 2012), 11-14
- Cesare Pautasso, Olaf Zimmermann, and Frank Leymann. 2008. *Restful web services vs. "big" web services: making the right architectural decision*. In Proceedings of the 17th international conference on World Wide Web (WWW '08). ACM, New York, NY, USA, 805-814
- *REST Easy: API Design, Evolution, and Connection*. WSO2 White Paper
- T. Erl et al. *SOA with REST - Principles, Patterns & Constraints for Building Enterprise Solutions with REST*. Prentice-Hall. 2013
- James Lewis and Martin Fowler's article - <http://martinfowler.com/microservices>
- Building Microservices. Sam Newmann. O'Reilly
- Martin Fowler's page on micro services
  - <http://martinfowler.com/microservices/>
- "Building micro-services", Sam Newman, O'Reilly
- Migrating to microservices - talk at Qcon by Adrian Cockcroft, Netflix
  - <http://www.infoq.com/presentations/migration-cloud-native>